

EKS: Scoping Multiple Users to a Namespace via a Kubernetes Group

Cluster: "test-cluster" · Namespace: "dev-team" · RoleBinding subject kind: Group

When several users need the same access to one namespace, bind RBAC to a Kubernetes **Group** once, then map each IAM identity into that group. Adding or removing a person later only touches the identity mapping — the Role and RoleBinding never change.

STEP 1 · RUN AS ADMIN

Create the namespace

```
kubectl create namespace dev-team
```

STEP 2 · RUN AS ADMIN

Create a Role and RoleBinding with subject kind Group

```
# dev-team-role.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev-team
  name: dev-team-full-access
rules:
- apiGroups: ["", "apps", "extensions", "batch"]
  resources: ["*"]
  verbs: ["*"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-team-binding
  namespace: dev-team
subjects:
- kind: Group
  name: dev-team          # Kubernetes-side group name
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: dev-team-full-access
  apiGroup: rbac.authorization.k8s.io
```

```
kubectl apply -f dev-team-role.yaml
```

Using `dev-team` as both the namespace name and the Kubernetes group name is fine — just keep in mind they are two separate objects (a Namespace and an RBAC Group identity) that happen to share a name.

STEP 3 · RUN AS ADMIN

Map each IAM user's identity to the Kubernetes group

Preferred: EKS access entries (if enabled on the cluster)

```
aws eks create-access-entry \
  --cluster-name test-cluster \
  --principal-arn arn:aws:iam::<ACCOUNT_ID>:user/user-a \
  --username user-a \
  --kubernetes-groups dev-team

aws eks create-access-entry \
  --cluster-name test-cluster \
  --principal-arn arn:aws:iam::<ACCOUNT_ID>:user/user-b \
  --username user-b \
  --kubernetes-groups dev-team

aws eks create-access-entry \
  --cluster-name test-cluster \
  --principal-arn arn:aws:iam::<ACCOUNT_ID>:user/user-c \
```

```
--username user-c \  
--kubernetes-groups dev-team
```

Leave these access entries policy-less — do not attach `AmazonEKSClusterAdminPolicy` or similar — so RBAC (Step 2) remains the sole authority over what the group can do.

Alternative: aws-auth ConfigMap (older clusters without access entries)

```
mapUsers: |  
- userarn: arn:aws:iam::<ACCOUNT_ID>:user/user-a  
  username: user-a  
  groups:  
    - dev-team  
- userarn: arn:aws:iam::<ACCOUNT_ID>:user/user-b  
  username: user-b  
  groups:  
    - dev-team  
- userarn: arn:aws:iam::<ACCOUNT_ID>:user/user-c  
  username: user-c  
  groups:  
    - dev-team
```

STEP 4 · RUN AS ADMIN

Each user's IAM permissions

Each of user-a, user-b, user-c also needs an IAM policy to reach the cluster at all — `eks:DescribeCluster` and `eks:AccessKubernetesApi`, scoped to the `test-cluster` ARN:

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "EksClusterConnect",  
      "Effect": "Allow",  
      "Action": [  
        "eks:DescribeCluster",  
        "eks:AccessKubernetesApi"  
      ],  
      "Resource": "arn:aws:eks:<REGION>:<ACCOUNT_ID>:cluster/test-cluster"  
    }  
  ]  
}
```

Attach this via an IAM Group (e.g. `test-cluster-connect-users`) so it's managed once for all three, rather than attaching it to each IAM user individually:

```
aws iam create-policy \  
  --policy-name test-cluster-connect \  
  --policy-document file://test-cluster-connect-policy.json  
  
aws iam create-group --group-name test-cluster-connect-users  
  
aws iam attach-group-policy \  
  --group-name test-cluster-connect-users \  
  --policy-arn arn:aws:iam::<ACCOUNT_ID>:policy/test-cluster-connect  
  
aws iam add-user-to-group --user-name user-a --group-name test-cluster-connect-users  
aws iam add-user-to-group --user-name user-b --group-name test-cluster-connect-users  
aws iam add-user-to-group --user-name user-c --group-name test-cluster-connect-users
```

STEP 5 · RUN AS EACH USER

Each user generates their own kubeconfig

```
aws eks update-kubeconfig --name test-cluster --region us-east-1 --alias test-cluster  
kubectl config set-context test-cluster --namespace=dev-team
```

What actually enforces the boundary

The `--namespace=dev-team` default is convenience only. The real restriction is the combination of three layers: the IAM policy (lets a user reach the cluster at all), the access-entry / aws-auth mapping (assigns them to the `dev-team` Kubernetes group), and the RoleBinding (grants that group access only inside the `dev-team` namespace). Anyone in the group hitting another namespace gets a "forbidden" error.

Things to double-check afterward

- **Cluster-scoped resources** — `kubectl get nodes`, `kubectl get namespaces`, `kubectl get pv` — aren't covered by this namespaced Role. Anyone in the group gets "forbidden" on these unless a ClusterRole is separately granted.
- **No overlapping broader access** — make sure none of these three users (or the IAM policy attached to them) picks up wider permissions elsewhere: another mapping, an access entry with a cluster-wide policy attached, an overly broad IAM policy, or accidental overlap with a built-in Kubernetes group like `system:masters`.